

Complete Training Manual: Flask Machine Learning API Deployment with GitHub and Render

Introduction

This manual teaches how to create, train, test, host, and deploy a Machine Learning API using Flask, GitHub, and Render.

You will learn:

- How Linear Regression works
- Why we save models as .pkl files
- How Flask APIs work
- How Git tracks code
- How GitHub stores repositories
- How Render deploys applications
- How clients communicate with APIs

Project Architecture

User Request → Flask API → Machine Learning Model → Prediction → JSON Response

The user sends data to the Flask API.
The API loads the machine learning model.
The model predicts the output.
The API returns JSON back to the client.

Project Structure

```
house_price_api/  
■■■ train_model.py  
■■■ app.py  
■■■ house_model.pkl  
■■■ requirements.txt  
■■■ .gitignore
```

Step 1: Install Python Packages

Install all required packages:

```
pip install flask pandas scikit-learn joblib gunicorn requests
```

Why each package is used:

- Flask → Creates API endpoints
- pandas → Handles datasets
- scikit-learn → Machine learning algorithms
- joblib → Saves and loads trained models
- gunicorn → Production web server
- requests → Tests APIs

Step 2: Create Machine Learning Model

File: train_model.py

Purpose:

Train a Linear Regression model that predicts house price using house size.

train_model.py Code

```
import pandas as pd  
from sklearn.linear_model import LinearRegression  
import joblib  
df = pd.DataFrame({
```

```

        "size": [1000, 1200, 1500, 1800, 2000],
        "price": [50, 60, 75, 90, 100]
    })

    x = df[["size"]]
    y = df[["price"]]

    model = LinearRegression()
    model.fit(x, y)

    joblib.dump(model, "house_model.pkl")

    print("Model saved successfully")

```

Line-by-Line Explanation

1. `import pandas as pd`
Imports pandas for table-like data handling.
2. `from sklearn.linear_model import LinearRegression`
Imports Linear Regression algorithm.
3. `import joblib`
Used to save/load machine learning models.
4. `df = pd.DataFrame(...)`
Creates training dataset.
5. `x = df[["size"]]`
Input feature.
6. `y = df[["price"]]`
Target/output column.
7. `model = LinearRegression()`
Creates ML algorithm object.
8. `model.fit(x, y)`
Trains the model.
9. `joblib.dump(...)`
Saves trained model into a .pkl file.
10. `print(...)`
Confirms successful training.

Understanding Linear Regression

Linear Regression tries to learn a relationship between input and output.

Formula:
 $y = mx + b$

Where:

- y = predicted output
- x = input
- m = slope
- b = intercept

The algorithm learns the best line from training data.

Step 3: Create Flask API

Flask converts the machine learning model into a web service.

Why APIs?

Because websites, mobile apps, and other applications can call the API remotely.

app.py Code

```

from flask import Flask, request, jsonify
import joblib

app = Flask(__name__)

model = joblib.load("house_model.pkl")

@app.route("/")
def home():
    return "House Price Prediction API"

@app.route("/predict", methods=["POST"])
def predict():

    data = request.json

    size = data["size"]

    prediction = model.predict([[size]])

    return jsonify({
        "house_size": size,
        "predicted_price": float(prediction[0])
    })

if __name__ == "__main__":
    app.run(debug=True)

```

Flask Line-by-Line Explanation

1. Flask
Creates web application.
2. request
Reads incoming client data.
3. jsonify
Converts Python dictionary into JSON.
4. app = Flask(__name__)
Creates Flask application object.
5. model = joblib.load(...)
Loads trained ML model.
6. @app.route("/")
Home page endpoint.
7. @app.route("/predict", methods=["POST"])
Prediction endpoint.
8. request.json
Reads JSON sent by client.
9. model.predict(...)
Runs ML prediction.
10. jsonify(...)
Returns JSON response.
11. app.run(debug=True)
Starts Flask development server.

What is JSON?

JSON means JavaScript Object Notation.

Example:

```

{
    "size":1700
}

```

JSON is the standard format used by APIs.

Step 4: Create requirements.txt

Generate requirements file:

```
pip freeze > requirements.txt
```

Why?

Render uses this file to install dependencies automatically.

Step 5: Run Flask Locally

Run:

```
python app.py
```

Output:

Running on http://127.0.0.1:5000

Open browser:

http://127.0.0.1:5000

Step 6: Test API Using Python

```
test_api.py
```

```
import requests
```

```
url = "http://127.0.0.1:5000/predict"
```

```
data = {  
    "size":1700  
}
```

```
response = requests.post(url, json=data)
```

```
print(response.json())
```

Expected Output:

```
{  
    "house_size":1700,  
    "predicted_price":85.0  
}
```

How Client and Server Communicate

Client sends:

POST /predict

Body:

```
{  
    "size":1700  
}
```

Server:

- Reads JSON
- Uses model.predict()
- Returns prediction

Response:

```
{  
    "predicted_price":85.0  
}
```

Step 7: Git Basics

Git is a version control system.

Why use Git?

- Track code changes
- Collaborate with teams
- Maintain history
- Push code to GitHub

Git Commands

`git init`
Creates Git repository.

`git add .`
Stages files.

`git commit -m "Initial Commit"`
Creates snapshot of code.

`git branch -M main`
Renames branch to main.

`git remote add origin <repo-url>`
Connects local project to GitHub.

`git push -u origin main`
Uploads code to GitHub.

GitHub Repository Creation

1. Login to GitHub.
2. Click New Repository.
3. Repository name:
house-price-api
4. Click Create Repository.
5. Copy repository URL.

Step 8: Deploy to Render

1. Login to Render.
2. Click New → Web Service.
3. Connect GitHub account.
4. Select repository.
5. Configure deployment.

Render Configuration

Build Command:
`pip install -r requirements.txt`

Start Command:
`gunicorn app:app`

Why Gunicorn?
Gunicorn is a production-ready WSGI server.
It is more stable than Flask's built-in server.

How Render Deployment Works

GitHub Repository
↓
Render Clones Repository
↓
Installs Packages
↓
Runs `gunicorn app:app`

↓
Public URL Generated

Production Testing

Example:

```
import requests

url = "https://your-app.onrender.com/predict"

data = {
    "size":1700
}

response = requests.post(url, json=data)

print(response.json())
```

Error You Encountered

You sent:

```
{
  "feature1":10,
  "feature2":20
}
```

But API expected:

```
{
  "size":1700
}
```

Therefore:

```
size = data["size"]
caused KeyError('size').

Render returned HTML error page.
Then response.json() failed with JSONDecodeError.
```

Improved API Validation

```
if "size" not in data:
    return jsonify({
        "error":"Please provide size"
    }), 400
```

This prevents server crashes.

Updating Existing Deployment

Modify files.

Then:

```
git add .
git commit -m "Updated API"
git push origin main
```

Render automatically redeploys updated code.

Real-World Extensions

The same architecture can host:

- Random Forest
- XGBoost
- Deep Learning
- LSTM Models
- NLP Models
- Computer Vision APIs

Only the model changes.
Deployment architecture remains similar.

Conclusion

You now understand:

- Machine Learning model training
- Saving/loading models
- Flask APIs
- JSON communication
- Git and GitHub workflow
- Render deployment
- Production API testing
- Debugging deployment issues

You now have a complete end-to-end Machine Learning deployment workflow.